INDEX

Exp. No.	Date	Name of the Experiment	Page No.	Remarks
1		Design and develop a reprogrammable embedded Computer using 8051 Microcontroller.	1-3	
2(a)		Configure timer control registers of 8051 and develop a program to generate delay.	4-6	
2(b)		Use of general purpose I/O port of two controllers for data transfer	7-10	
3(a)		Simulate Binary Counter (8bits) on LEDs	11-13	
3(b)		To interface 8 LEDs at I/O ports and create different patterns.	14-16	
4(a)		Serial I/O Configure 8051 Serial port for Asynchronous Serial Communication.	17-19	
4(b)		To demonstrate seven segments LED display and generate counting from 0 to 99.	20-22	
5		Develop a program to interface stepper motors with 8051.	23-25	
6		Generate traffic signals.	26-28	

Exprement No. : 01

Problem Name: Design and develop a reprogrammable embedded Computer using 8051 Microcontroller.

Apparatus:

1. Computer
2. Keil uVision5 software
3. Proteus 8 Professional software
4. AT89C51
5. LED

Procedure:

For Arm Keil:

1. Open Keil µVision, go to Project → New µVision Project, give a name and choose AT89C51 (Legacy Device Database). Click No for the Startup file.
2. Add a New file, save it with .c extension and add it to 'Source Group1'.
3. Options for Target → Output tab → tick 'Create HEX File'.
4. Write the code and build the project (F7). The hex file is created in the Objects folder.

For Proteus:

1. Open Proteus ISIS, click P (Pick From Libraries) and add all the required components.
2. Place the components on the workspace and wire up the circuit.
3. Load the hex file in AT89C51 (Program File) and click the Play button to start simulation.

Source Code:

```
#include<reg52.h>

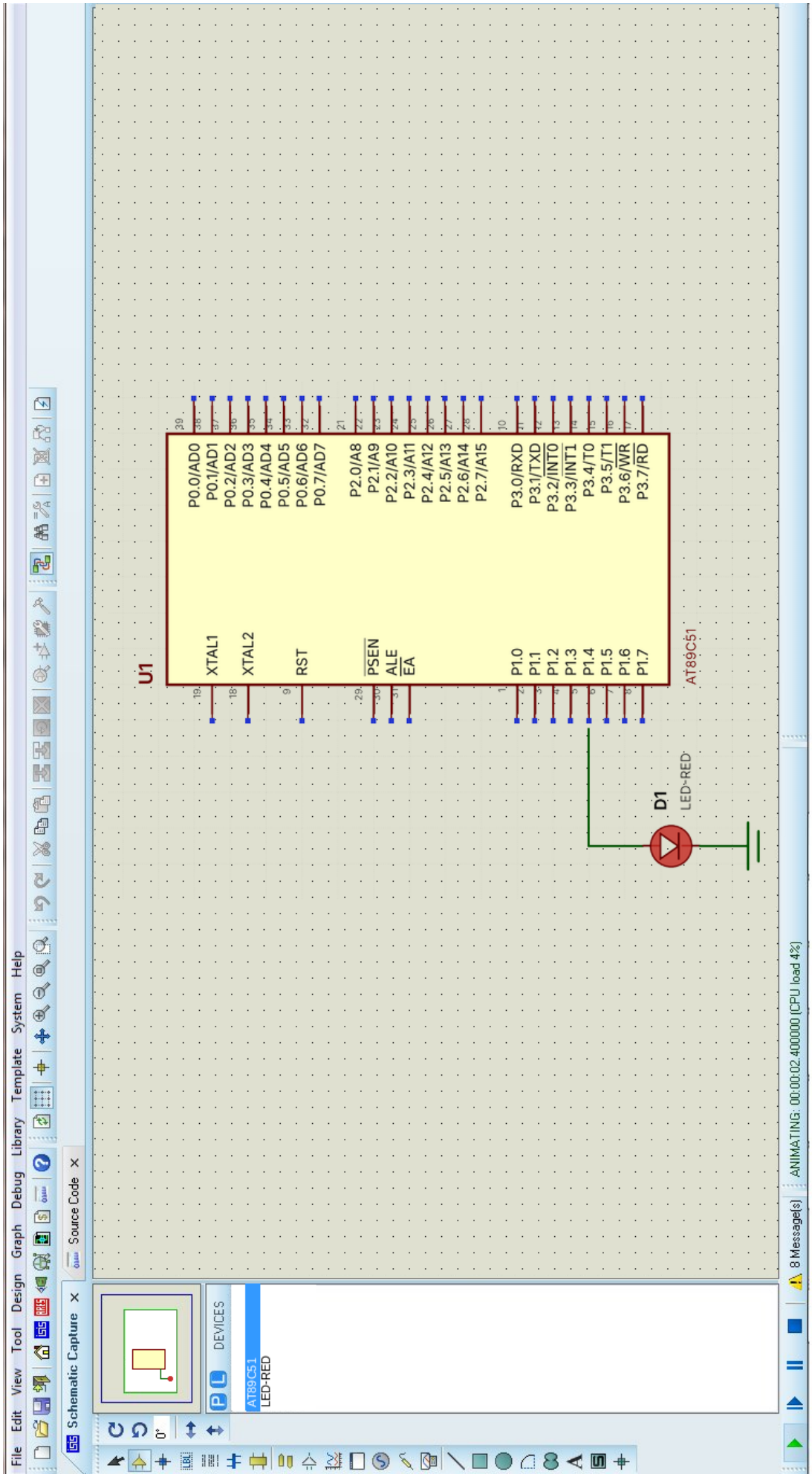
sbit LED = P1^4;
void Delay(void);

void main(void)
{
   while(1)
   {
      LED = 1;
      Delay();
      LED = 0;
      Delay();
   }
}




void Delay(void)
{
   unsigned int i;
   unsigned int j;
   for(i = 0; i < 8; i++)
   {
      for(j = 0; j < 12000; j++)
      {
      }
   }
}
```

Output:


File Edit View Tool Design Graph Debug Library Template System Help
Schematic Capture
Source Code
DEVICES
AT89C51
LED-RED
U1
XTAL1
XTAL2
RST
PSEN
ALE
EA
P0.0/AD0
P0.1/AD1
P0.2/AD2
P0.3/AD3
P0.4/AD4
P0.5/AD5
P0.6/AD6
P0.7/AD7
P2.0/A8
P2.1/A9
P2.2/A10
P2.3/A11
P2.4/A12
P2.5/A13
P2.6/A14
P2.7/A15
P3.0/RXD
P3.1/TXD
P3.2/INT0
P3.3/INT1
P3.4/T0
P3.5/T1
P3.6/WR
P3.7/RD
P1.0
P1.1
P1.2
P1.3
P1.4
P1.5
P1.6
P1.7
AT89C51
D1
LED-RED
8 Message(s)
ANIMATING: 00:00:02.400000 (CPU load 4%)

Exprement No. : 02(a)

Problem Name: Configure timer control registers of 8051 and develop a program to generate delay.

Apparatus:

1. Computer
2. Keil uVision5 software
3. Proteus 8 Professional software
4. AT89C51
5. LED

Procedure:

For Arm Keil:

1. Open Keil µVision, go to Project → New µVision Project, give a name and choose AT89C51 (Legacy Device Database). Click No for the Startup file.
2. Add a New file, save it with .c extension and add it to 'Source Group1'.
3. Options for Target → Output tab → tick 'Create HEX File'.
4. Write the code and build the project (F7). The hex file is created in the Objects folder.

For Proteus:

1. Open Proteus ISIS, click P (Pick From Libraries) and add all the required components.
2. Place the components on the workspace and wire up the circuit.
3. Load the hex file in AT89C51 (Program File) and click the Play button to start simulation.

Source Code:

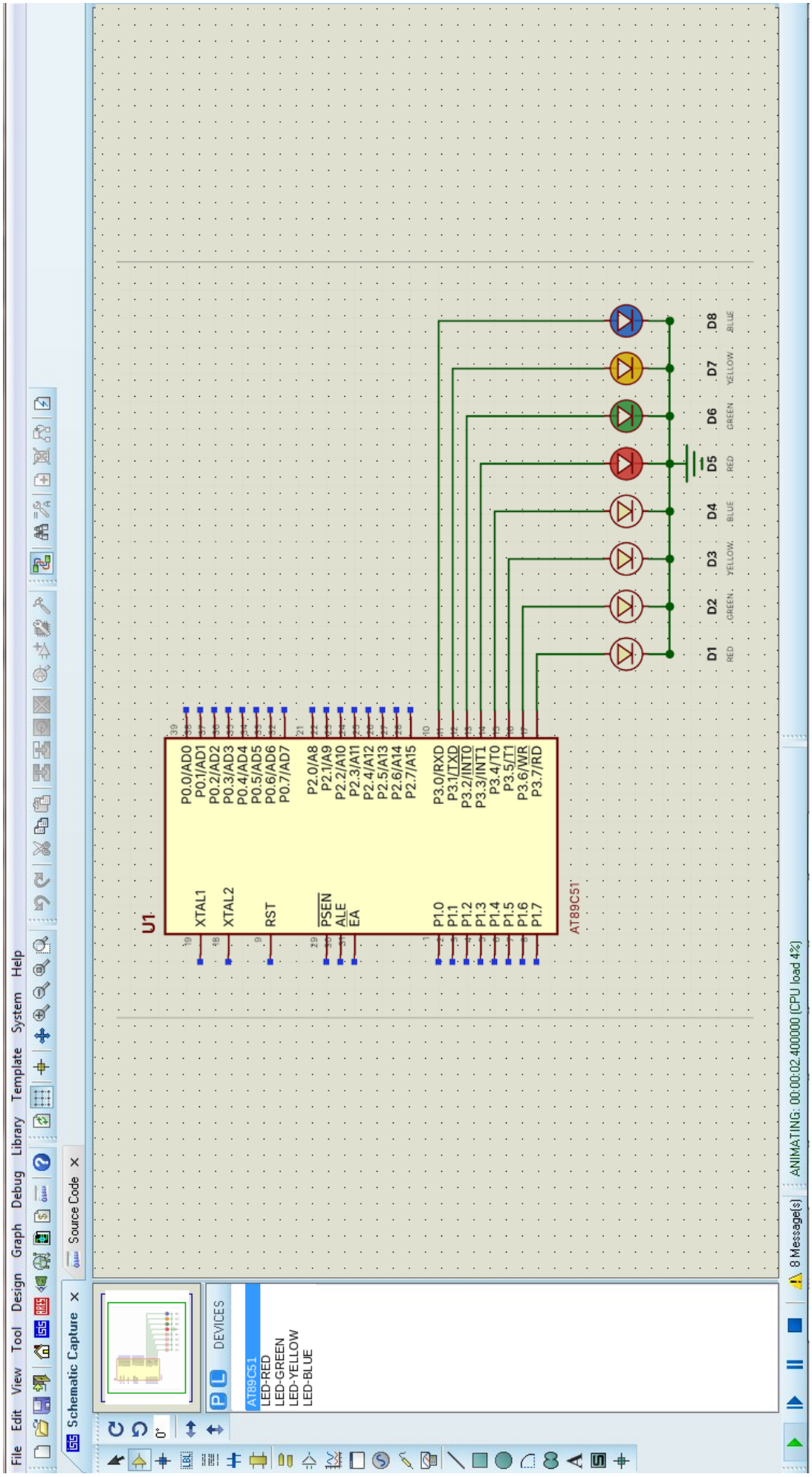
```
#include<reg51.h>

void Delay(unsigned char count);
                //timer delay function

void main(void)
{
  TMOD = 0x01;  //Timer 0, mode 1 (16-bit)
  while(1)
  {
    P3 = 0x0F;  //low nibble ON
    Delay(10);  //10 x 50 ms = 0.5 s
    P3 = 0xF0;  //high nibble ON
    Delay(10);
  }
}
```

```
void Delay(unsigned char count)
{
  unsigned char n;
  for(n = 0; n < count; n++)
  {
    TH0 = 0x3C;  //50 ms at 12 MHz
    TL0 = 0xB0;
    TR0 = 1;    //start Timer 0
    while(TF0 == 0);  //wait overflow
    TR0 = 0;    //stop Timer 0
    TF0 = 0;    //clear flag
  }
}
```

Output:


File Edit View Tool Design Graph Debug Library Template System Help
Schematic Capture
Source Code
DEVICES
AT89C51
LED-RED
LED-GREEN
LED-YELLOW
LED-BLUE
U1
XTAL1
XTAL2
RST
PSEN
ALE
EA
P0.0/AD0
P0.1/AD1
P0.2/AD2
P0.3/AD3
P0.4/AD4
P0.5/AD5
P0.6/AD6
P0.7/AD7
P2.0/A8
P2.1/A9
P2.2/A10
P2.3/A11
P2.4/A12
P2.5/A13
P2.6/A14
P2.7/A15
P1.0
P1.1
P1.2
P1.3
P1.4
P1.5
P1.6
P1.7
P3.0/RXD
P3.1/TXD
P3.2/INT0
P3.3/INT1
P3.4/T0
P3.5/T1
P3.6/WR
P3.7/RD
AT89C51
D1 RED
D2 GREEN
D3 YELLOW
D4 BLUE
D5 RED
D6 GREEN
D7 YELLOW
D8 BLUE
8 Message(s)
ANIMATING: 00:00:02.400000 (CPU load 4%)

Exprement No.: 02(b)

Problem Name: Use of general purpose I/O port of two controllers for data transfer

Apparatus:

1. Computer
2. Keil uVision5 software
3. Proteus 8 Professional software
4. AT89C51
5. LED

Procedure:

For Arm Keil:

1. Open Keil µVision, go to Project → New µVision Project, give a name and choose AT89C51 (Legacy Device Database). Click No for the Startup file.
2. Add a New file, save it with .c extension and add it to 'Source Group1'.
3. Options for Target → Output tab → tick 'Create HEX File'.
4. Write the code and build the project (F7). The hex file is created in the Objects folder.

For Proteus:

1. Open Proteus ISIS, click P (Pick From Libraries) and add all the required components.
2. Place the components on the workspace and wire up the circuit.
3. Load the hex file in AT89C51 (Program File) and click the Play button to start simulation.

Source Code:

Code: (1)

```
//U1 : transmitter, data goes out on Port 3
#include<reg51.h>

void Delay(unsigned int time);
//delay function with time value

void main(void)
{
   unsigned char k;
   while(1)
   {
      for(k = 0; k < 8; k++)
      {
         P3 = (unsigned char)(0x01 << k);
         Delay(500);
      }
   }
}
```

```
void Delay(unsigned int time)
{
   unsigned int i, j;
   for(i = 0; i < time; i++)
   {
      for(j = 0; j < 120; j++)
      {
      }
   }
}
```

Code: (2)

```
//U2 : receiver, data comes in on Port 1
#include<reg51.h>

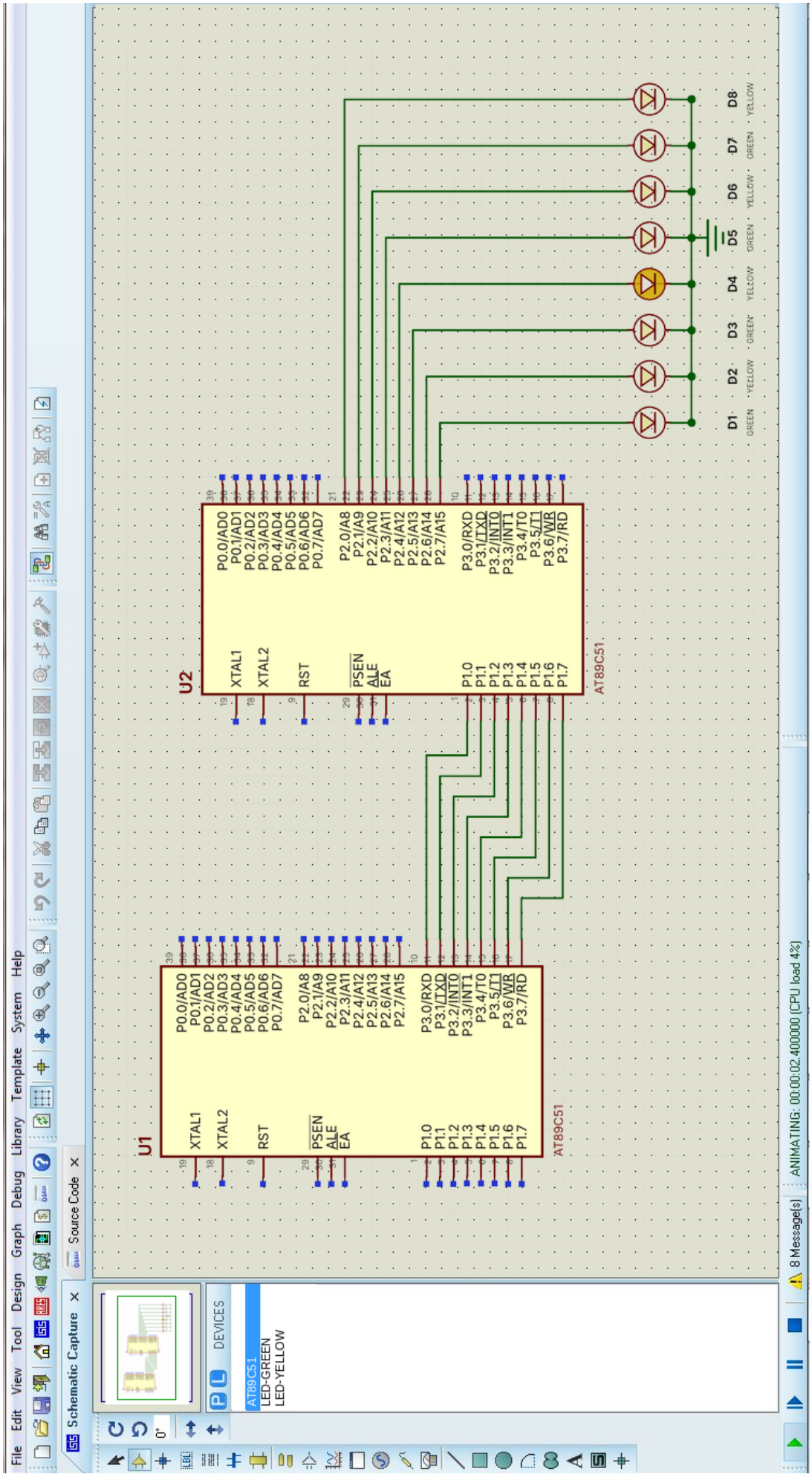
void Delay(unsigned int time);
//delay function with time value

void main(void)
{
   P1 = 0xFF;    //make Port 1 an input port
   while(1)
   {
      P2 = P1;    //copy received data to LEDs
      Delay(100);
   }
}




void Delay(unsigned int time)
{
   unsigned int i, j;
   for(i = 0; i < time; i++)
   {
      for(j = 0; j < 120; j++)
      {
      }
   }
}
```

Output:


File Edit View Tool Design Graph Debug Library Template System Help
Schematic Capture
Source Code
DEVICES
AT89C51
LED-GREEN
LED-YELLOW
U1
U2
XTAL1
XTAL2
RST
PSEN
ALE
EA
P0.0/AD0
P0.1/AD1
P0.2/AD2
P0.3/AD3
P0.4/AD4
P0.5/AD5
P0.6/AD6
P0.7/AD7
P2.0/A8
P2.1/A9
P2.2/A10
P2.3/A11
P2.4/A12
P2.5/A13
P2.6/A14
P2.7/A15
P3.0/RXD
P3.1/TXD
P3.2/INT0
P3.3/INT1
P3.4/T0
P3.5/T1
P3.6/WR
P3.7/RD
P1.0
P1.1
P1.2
P1.3
P1.4
P1.5
P1.6
P1.7
AT89C51
D1 GREEN
D2 YELLOW
D3 GREEN
D4 YELLOW
D5 GREEN
D6 YELLOW
D7 GREEN
D8 YELLOW
8 Message(s)
ANIMATING: 00:00:02.400000 (CPU load 4%)

Exprement No. : 03(a)

Problem Name: Simulate Binary Counter (8bits) on LEDs

Apparatus:

1. Computer
2. Keil uVision5 software
3. Proteus 8 Professional software
4. AT89C51
5. LED

Procedure:

For Arm Keil:

1. Open Keil µVision, go to Project → New µVision Project, give a name and choose AT89C51 (Legacy Device Database). Click No for the Startup file.
2. Add a New file, save it with .c extension and add it to 'Source Group1'.
3. Options for Target → Output tab → tick 'Create HEX File'.
4. Write the code and build the project (F7). The hex file is created in the Objects folder.

For Proteus:

1. Open Proteus ISIS, click P (Pick From Libraries) and add all the required components.
2. Place the components on the workspace and wire up the circuit.
3. Load the hex file in AT89C51 (Program File) and click the Play button to start simulation.

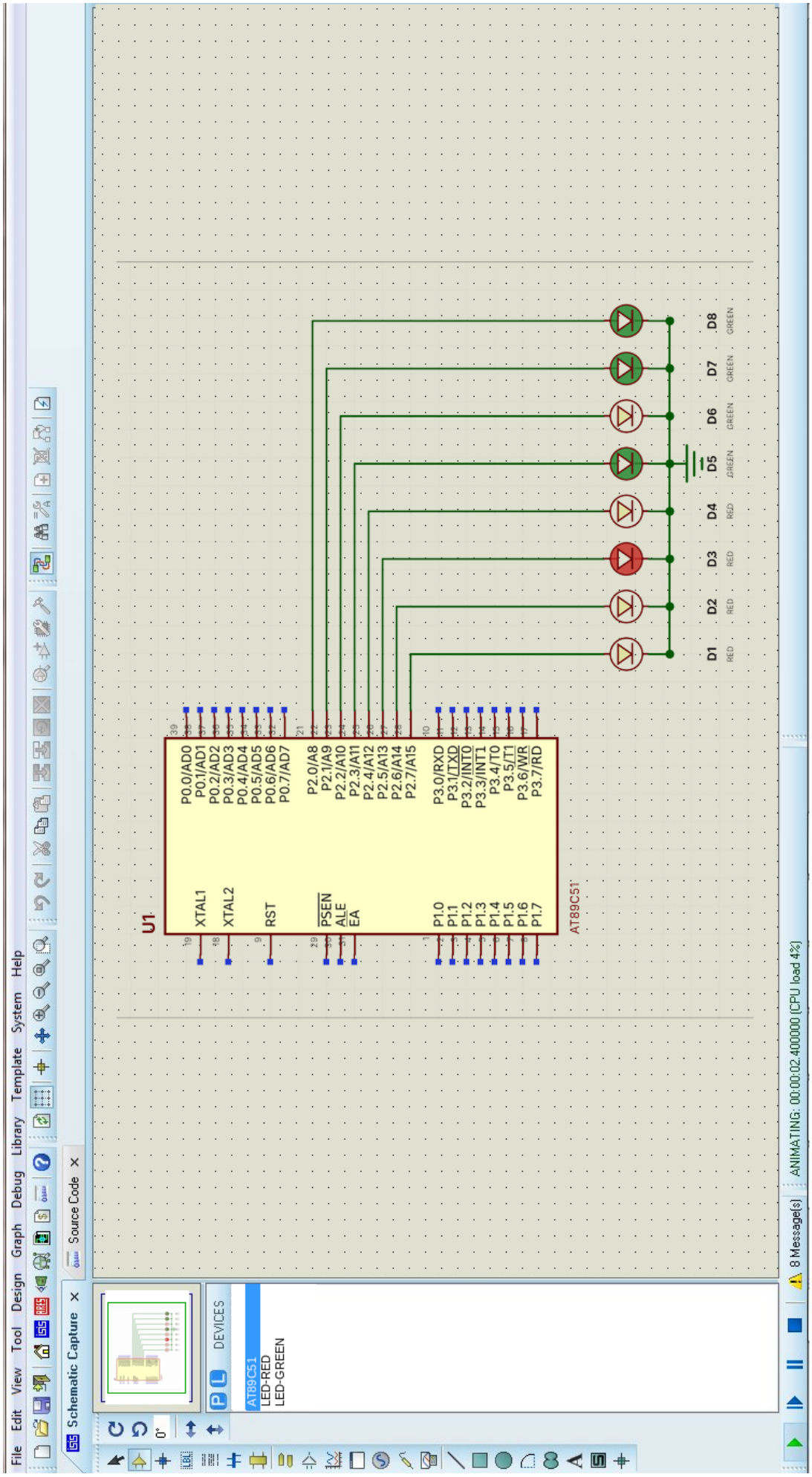
Source Code:

```
#include<reg51.h>
void delay(unsigned int time);

void main()
{
   P2 = 0x00;      //all bits of P2 set LOW
   while(1)
   {
      P2++;       //increment of P2
      delay(300);
   }
}

void delay(unsigned int time)
{
   unsigned int i, j;
   for(i = 0; i < time; i++)
   {
      for(j = 0; j < 120; j++)
      {
      }
   }
}
```

Output:


File Edit View Tool Design Graph Debug Library Template System Help
Schematic Capture
Source Code
DEVICES
AT89C51
LED-RED
LED-GREEN
U1
XTAL1
XTAL2
RST
PSEN
ALE
EA
P0.0/AD0
P0.1/AD1
P0.2/AD2
P0.3/AD3
P0.4/AD4
P0.5/AD5
P0.6/AD6
P0.7/AD7
P2.0/A8
P2.1/A9
P2.2/A10
P2.3/A11
P2.4/A12
P2.5/A13
P2.6/A14
P2.7/A15
P3.0/RXD
P3.1/TXD
P3.2/INT0
P3.3/INT1
P3.4/T0
P3.5/T1
P3.6/WR
P3.7/RD
P1.0
P1.1
P1.2
P1.3
P1.4
P1.5
P1.6
P1.7
AT89C51
D1 RED
D2 RED
D3 RED
D4 RED
D5 GREEN
D6 GREEN
D7 GREEN
D8 GREEN
8 Message(s)
ANIMATING: 00:00:02.400000 (CPU load 4%)

Exprement No. : 03(b)

Problem Name: To interface 8 LEDs at I/O ports and create different patterns.

Apparatus:

1. Computer
2. Keil uVision5 software
3. Proteus 8 Professional software
4. AT89C51
5. LED

Procedure:

For Arm Keil:

1. Open Keil µVision, go to Project → New µVision Project, give a name and choose AT89C51 (Legacy Device Database). Click No for the Startup file.
2. Add a New file, save it with .c extension and add it to 'Source Group1'.
3. Options for Target → Output tab → tick 'Create HEX File'.
4. Write the code and build the project (F7). The hex file is created in the Objects folder.

For Proteus:

1. Open Proteus ISIS, click P (Pick From Libraries) and add all the required components.
2. Place the components on the workspace and wire up the circuit.
3. Load the hex file in AT89C51 (Program File) and click the Play button to start simulation.

Source Code:

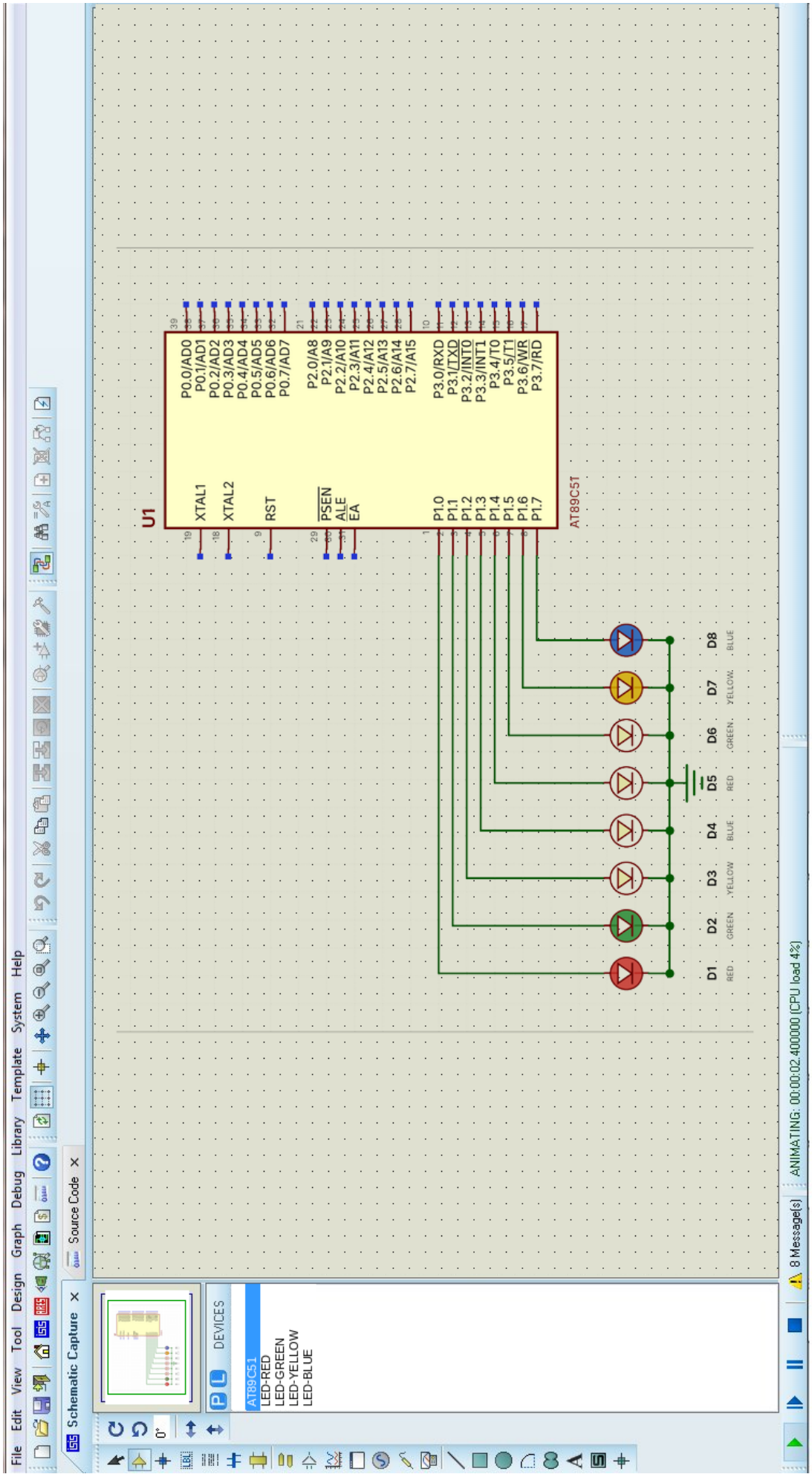
```
#include<reg51.h>

unsigned char code pattern[8] =
   {0x81, 0xC3, 0xE7, 0xFF,
    0xE7, 0xC3, 0x81, 0x00};

void delay(unsigned int time)
{
   unsigned int i, j;
   for(i = 0; i < time; i++)
   {
      for(j = 0; j < 120; j++)
      {
      }
   }
}

void main()
{
   unsigned char k;
   while(1)
   {
      for(k = 0; k < 8; k++)
      {
         P1 = pattern[k];
         delay(400);
      }
      for(k = 0; k < 8; k++)
      {
         P1 = 0x01 << k;  //chaser
         delay(200);
      }
   }
}
```

Output:


File Edit View Tool Design Graph Debug Library Template System Help
Schematic Capture
Source Code
DEVICES
AT89C51
LED-RED
LED-GREEN
LED-YELLOW
LED-BLUE
U1
XTAL1
XTAL2
RST
PSEN
ALE
EA
P0.0/AD0
P0.1/AD1
P0.2/AD2
P0.3/AD3
P0.4/AD4
P0.5/AD5
P0.6/AD6
P0.7/AD7
P2.0/A8
P2.1/A9
P2.2/A10
P2.3/A11
P2.4/A12
P2.5/A13
P2.6/A14
P2.7/A15
P3.0/RXD
P3.1/TXD
P3.2/INT0
P3.3/INT1
P3.4/T0
P3.5/T1
P3.6/WR
P3.7/RD
P1.0
P1.1
P1.2
P1.3
P1.4
P1.5
P1.6
P1.7
AT89C51
D1 RED
D2 GREEN
D3 YELLOW
D4 BLUE
D5 RED
D6 GREEN
D7 YELLOW
D8 BLUE
8 Message(s)
ANIMATING: 00:00:02.400000 (CPU load 4%)

Exprement No.: 04(a)

Problem Name: Serial I/O Configure 8051 Serial port for Asynchronous Serial Communication.

Apparatus:

1. Computer
2. Keil uVision5 software
3. Proteus 8 Professional software
4. AT89C51
5. LED

Procedure:

For Arm Keil:

1. Open Keil µVision, go to Project → New µVision Project, give a name and choose AT89C51 (Legacy Device Database). Click No for the Startup file.
2. Add a New file, save it with .c extension and add it to 'Source Group1'.
3. Options for Target → Output tab → tick 'Create HEX File'.
4. Write the code and build the project (F7). The hex file is created in the Objects folder.

For Proteus:

1. Open Proteus ISIS, click P (Pick From Libraries) and add all the required components.
2. Place the components on the workspace and wire up the circuit.
3. Load the hex file in AT89C51 (Program File) and click the Play button to start simulation.

Source Code:

```
#include<reg51.h>
void sendChar(char x);
void sendString(char *str);

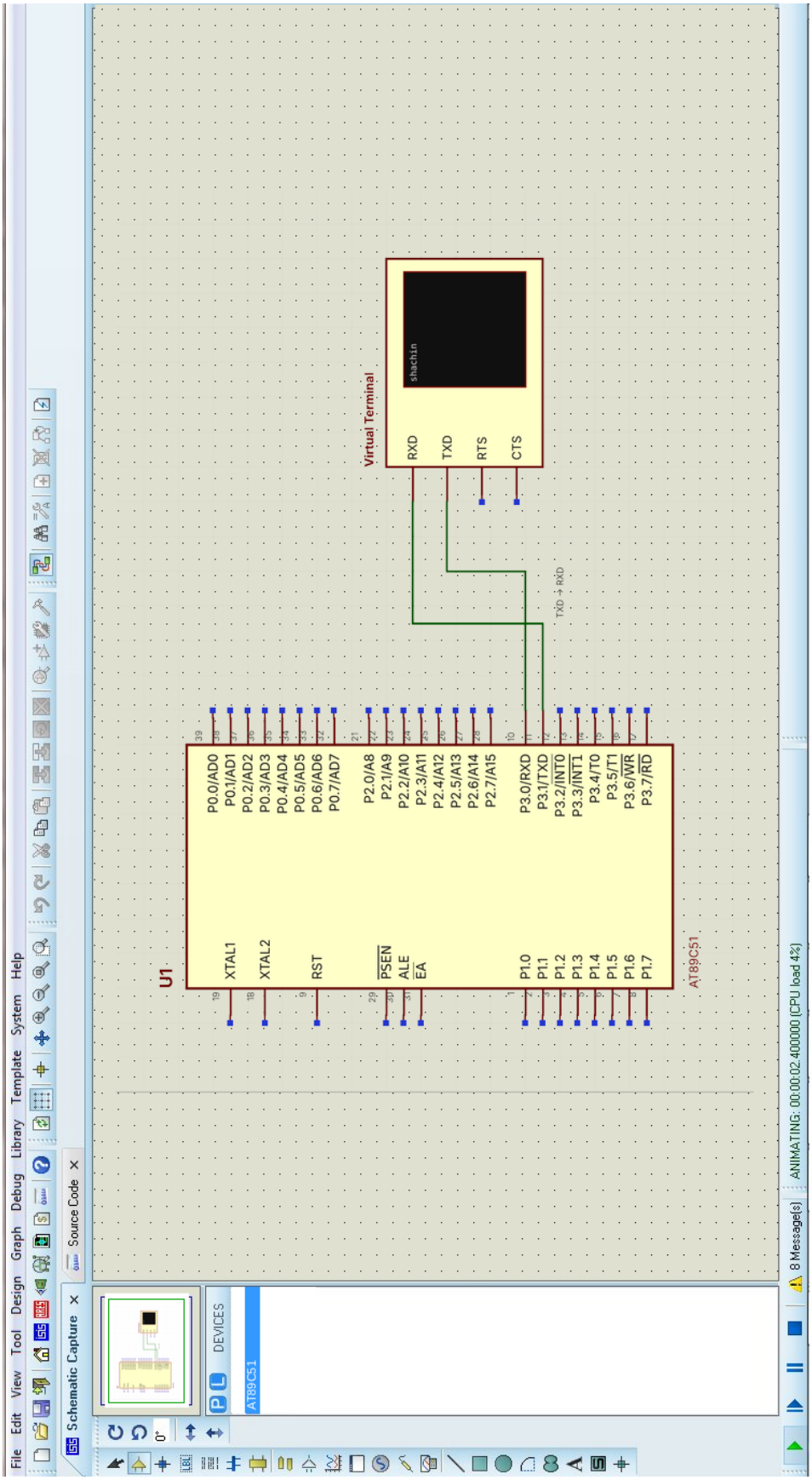
void main(void)
{
   TMOD = 0x20;   //Timer 1, auto reload
   TH1 = 0xFD;    //9600 baud rate
   SCON = 0x50;   //Mode 1, reception enable
   TR1 = 1;       //Start timer 1

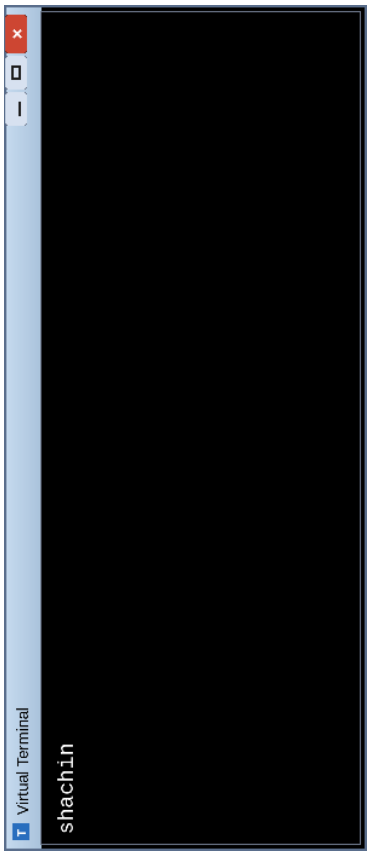
   sendString("shachin");
   sendChar('\r');   //carriage return
   sendChar('\n');   //new line
   while(1);
}
```

```
void sendChar(char x)
{
   SBUF = x;
   while(TI == 0);
   TI = 0;
}

void sendString(char *str)
{
   while(*str != '\0')
   {
      sendChar(*str);
      str++;
   }
}
```

Output:


File Edit View Tool Design Graph Debug Library Template System Help
Schematic Capture
Source Code
DEVICES
AT89C51
U1
XTAL1
XTAL2
RST
PSEN
ALE
EA
P0.0/AD0
P0.1/AD1
P0.2/AD2
P0.3/AD3
P0.4/AD4
P0.5/AD5
P0.6/AD6
P0.7/AD7
P2.0/A8
P2.1/A9
P2.2/A10
P2.3/A11
P2.4/A12
P2.5/A13
P2.6/A14
P2.7/A15
P3.0/RXD
P3.1/TXD
P3.2/INT0
P3.3/INT1
P3.4/T0
P3.5/T1
P3.6/WR
P3.7/RD
P1.0
P1.1
P1.2
P1.3
P1.4
P1.5
P1.6
P1.7
AT89C51
Virtual Terminal
RXD
TXD
RTS
CTS
shachin
TXD -> RXD
8 Message(s)
ANIMATING: 00:00:02.400000 (CPU load 4%)


Virtual Terminal
shachin

Exprement No.: 04(b)

Problem Name: To demonstrate seven segments LED display and generate counting from 0 to 99.

Apparatus:

1. Computer
2. Keil uVision5 software
3. Proteus 8 Professional software
4. AT89C51
5. LED

Procedure:

For Arm Keil:

1. Open Keil µVision, go to Project → New µVision Project, give a name and choose AT89C51 (Legacy Device Database). Click No for the Startup file.
2. Add a New file, save it with .c extension and add it to 'Source Group1'.
3. Options for Target → Output tab → tick 'Create HEX File'.
4. Write the code and build the project (F7). The hex file is created in the Objects folder.

For Proteus:

1. Open Proteus ISIS, click P (Pick From Libraries) and add all the required components.
2. Place the components on the workspace and wire up the circuit.
3. Load the hex file in AT89C51 (Program File) and click the Play button to start simulation.

Source Code:

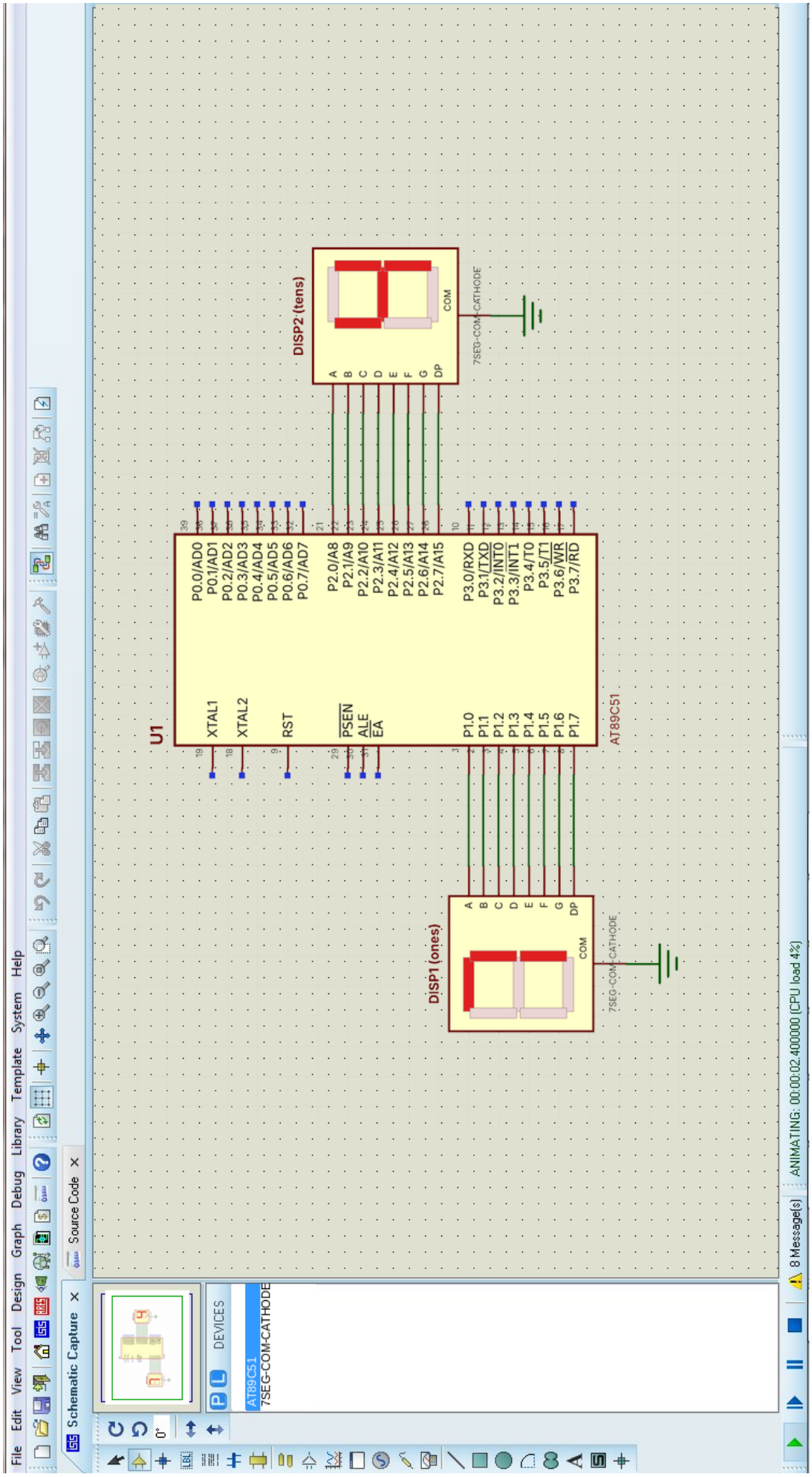
```
#include<reg51.h>

//hex values of digits 0-9 for 7-seg LED
//(common cathode)
unsigned char code digit[10] =
   {0x3F, 0x06, 0x5B, 0x4F, 0x66,
    0x6D, 0x7D, 0x07, 0x7F, 0x6F};

void delay(unsigned int ms)
{
   unsigned int i, j;
   for(i = 0; i < ms; i++)
   {
      for(j = 0; j < 120; j++)
      {
      }
   }
}

void main(void)
{
   unsigned char count;
   P1 = 0x00;
   P2 = 0x00;
   while(1)
   {
      for(count = 0; count <= 99; count++)
      {
         P2 = digit[count / 10];  //tens
         P1 = digit[count % 10];  //ones
         delay(500);
      }
   }
}
```

Output:


File Edit View Tool Design Graph Debug Library Template System Help
Schematic Capture
Source Code
DEVICES
AT89C51
7SEG-COM-CATHODE
U1
AT89C51
XTAL1
XTAL2
RST
PSEN
ALE
EA
P0.0/AD0
P0.1/AD1
P0.2/AD2
P0.3/AD3
P0.4/AD4
P0.5/AD5
P0.6/AD6
P0.7/AD7
P2.0/A8
P2.1/A9
P2.2/A10
P2.3/A11
P2.4/A12
P2.5/A13
P2.6/A14
P2.7/A15
P3.0/RXD
P3.1/TXD
P3.2/INT0
P3.3/INT1
P3.4/T0
P3.5/T1
P3.6/WR
P3.7/RD
P1.0
P1.1
P1.2
P1.3
P1.4
P1.5
P1.6
P1.7
DISP2 (tens)
DISP1 (ones)
A
B
C
D
E
F
G
DP
COM
7SEG-COM-CATHODE
8 Message(s)
ANIMATING: 00:00:02.400000 (CPU load 4%)

Exprement No. : 05

Problem Name: Develop a program to interface stepper motors with 8051.

Apparatus:

1. Computer
2. Keil uVision5 software
3. Proteus 8 Professional software
4. AT89C51
5. ULN2003A driver
6. Stepper motor

Procedure:

For Arm Keil:

1. Open Keil µVision, go to Project → New µVision Project, give a name and choose AT89C51 (Legacy Device Database). Click No for the Startup file.
2. Add a New file, save it with .c extension and add it to 'Source Group1'.
3. Options for Target → Output tab → tick 'Create HEX File'.
4. Write the code and build the project (F7). The hex file is created in the Objects folder.

For Proteus:

1. Open Proteus ISIS, click P (Pick From Libraries) and add all the required components.
2. Place the components on the workspace and wire up the circuit.
3. Load the hex file in AT89C51 (Program File) and click the Play button to start simulation.

Source Code:

```
#include<reg51.h>

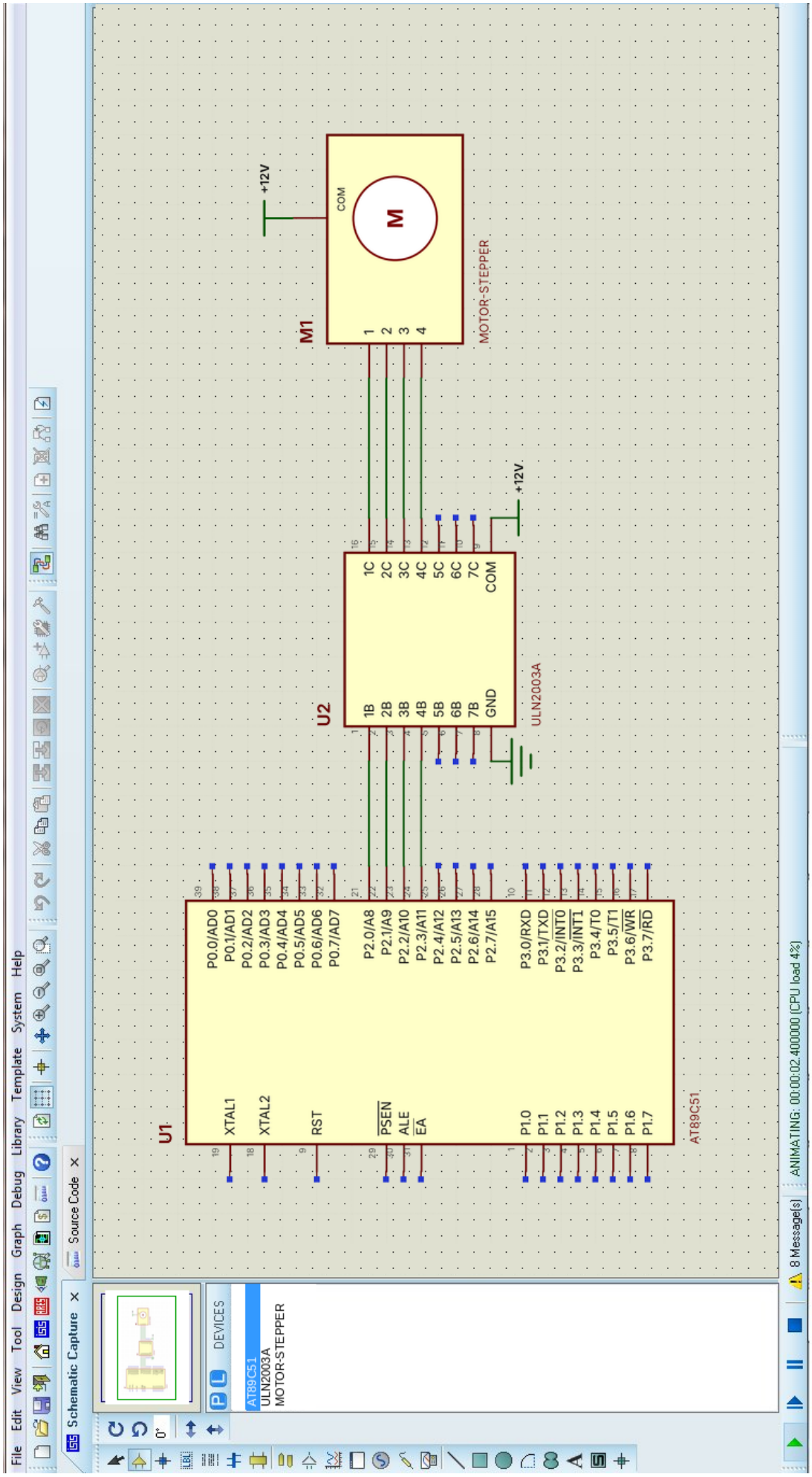
//full step sequence of the motor
unsigned char code step[4] =
  {0x03, 0x06, 0x0C, 0x09};

void delay(unsigned int ms)
{
  unsigned int i, j;
  for(i = 0; i < ms; i++)
  {
    for(j = 0; j < 120; j++)
    {
    }
  }
}




void main(void)
{
  unsigned int n;
  while(1)
  {
    for(n = 0; n < 200; n++)
    {
      P2 = step[n % 4];     //CW
      delay(20);
    }
    delay(1000);
    for(n = 0; n < 200; n++)
    {
      P2 = step[3 - (n % 4)]; //ACW
      delay(20);
    }
    delay(1000);
  }
}
```

Output:


File Edit View Tool Design Graph Debug Library Template System Help
Schematic Capture
Source Code
DEVICES
AT89C51
ULN2003A
MOTOR-STEPPER
U1
U2
M1
+12V
COM
M
MOTOR-STEPPER
ULN2003A
AT89C51
XTAL1
XTAL2
RST
PSEN
ALE
EA
P0.0/AD0
P0.1/AD1
P0.2/AD2
P0.3/AD3
P0.4/AD4
P0.5/AD5
P0.6/AD6
P0.7/AD7
P2.0/A8
P2.1/A9
P2.2/A10
P2.3/A11
P2.4/A12
P2.5/A13
P2.6/A14
P2.7/A15
P3.0/RXD
P3.1/TXD
P3.2/INT0
P3.3/INT1
P3.4/T0
P3.5/T1
P3.6/WR
P3.7/RD
P1.0
P1.1
P1.2
P1.3
P1.4
P1.5
P1.6
P1.7
1B
2B
3B
4B
5B
6B
7B
GND
1C
2C
3C
4C
5C
6C
7C
COM
8 Message(s)
ANIMATING: 00:00:02.400000 (CPU load 4%)

Exprement No. : 06

Problem Name: Generate traffic signals.

Apparatus:

1. Computer
2. Keil uVision5 software
3. Proteus 8 Professional software
4. AT89C51
5. LED (red, yellow, green)

Procedure:

For Arm Keil:

1. Open Keil µVision, go to Project → New µVision Project, give a name and choose AT89C51 (Legacy Device Database). Click No for the Startup file.
2. Add a New file, save it with .c extension and add it to 'Source Group1'.
3. Options for Target → Output tab → tick 'Create HEX File'.
4. Write the code and build the project (F7). The hex file is created in the Objects folder.

For Proteus:

1. Open Proteus ISIS, click P (Pick From Libraries) and add all the required components.
2. Place the components on the workspace and wire up the circuit.
3. Load the hex file in AT89C51 (Program File) and click the Play button to start simulation.

Source Code:

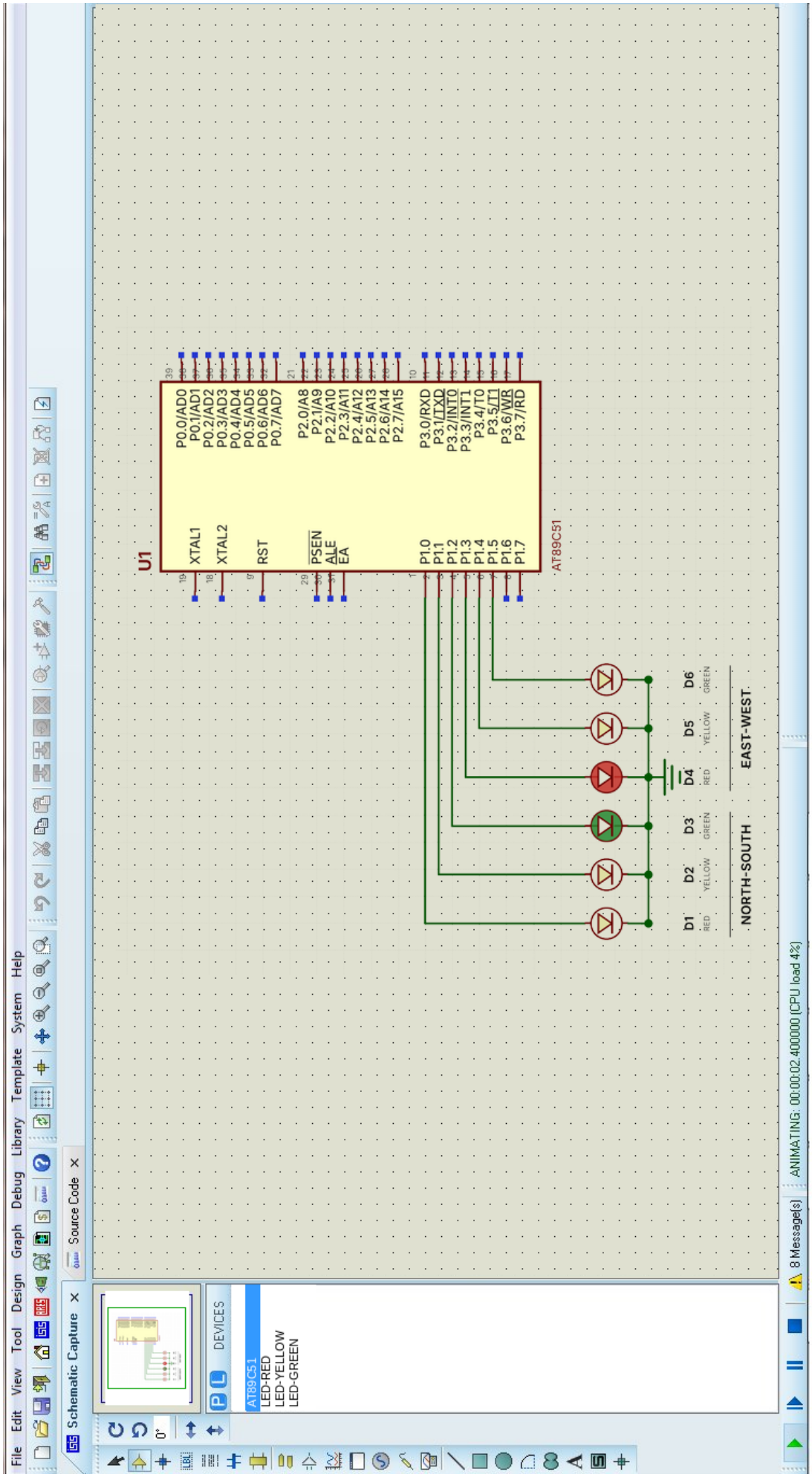
```
#include<reg51.h>

void delay(unsigned int ms)
{
   unsigned int i, j;
   for(i = 0; i < ms; i++)
   {
      for(j = 0; j < 120; j++)
      {
      }
   }
}




void main(void)
{
   while(1)
   {
      P1 = 0x0C;   //NS green, EW red
      delay(5000);
      P1 = 0x0A;   //NS yellow, EW red
      delay(2000);
      P1 = 0x21;   //NS red, EW green
      delay(5000);
      P1 = 0x11;   //NS red, EW yellow
      delay(2000);
   }
}
```

Output:


File Edit View Tool Design Graph Debug Library Template System Help
Schematic Capture
Source Code
DEVICES
AT89C51
LED-RED
LED-YELLOW
LED-GREEN
U1
AT89C51
XTAL1
XTAL2
RST
PSEN
ALE
EA
P0.0/AD0
P0.1/AD1
P0.2/AD2
P0.3/AD3
P0.4/AD4
P0.5/AD5
P0.6/AD6
P0.7/AD7
P2.0/A8
P2.1/A9
P2.2/A10
P2.3/A11
P2.4/A12
P2.5/A13
P2.6/A14
P2.7/A15
P3.0/RXD
P3.1/TXD
P3.2/INT0
P3.3/INT1
P3.4/T0
P3.5/T1
P3.6/WR
P3.7/RD
P1.0
P1.1
P1.2
P1.3
P1.4
P1.5
P1.6
P1.7
D1 RED
D2 YELLOW
D3 GREEN
D4 RED
D5 YELLOW
D6 GREEN
NORTH-SOUTH
EAST-WEST
8 Message(s)
ANIMATING: 00:00:02.400000 (CPU load 4%)